



Procesamiento Del Lenguaje Natural

Trabajo Práctico Integrador

Informe del proyecto

Integrantes:

- Juan Ignacio Pintarelli
- Juan Cruz Rey
- Mateo Chiza
- Paula Goldenhorn

Número de grupo: 09

Fecha de entrega: 03/06/2026

Modalidad: Asincrónico

Docente: Victor Ezequiel Muñoz

Informe del proyecto

1. Introducción

En entornos educativos, especialmente en clases presenciales, es frecuente que surjan incidencias operativas que interrumpen el desarrollo normal de la actividad. Entre los problemas más comunes se encuentran la falta de tinta en marcadores para pizarra, fallas en computadoras, problemas con el aire acondicionado, entre otras. Estas situaciones generan demoras, ya que el docente debe interrumpir la clase, buscar asistencia en recepción u otra área, volver al aula y esperar la resolución del inconveniente.

El problema que aborda este proyecto es la necesidad de reducir ese tiempo de respuesta mediante un sistema que permita al docente registrar rápidamente la incidencia en lenguaje natural y que, a partir de ese mensaje, se genere automáticamente un ticket estructurado con la información relevante. Por ejemplo, ante un mensaje como *“no tengo marcadores en el aula 4”*, el sistema debería identificar el objeto afectado, el lugar y el problema, y producir una salida estructurada lista para su gestión.

El enfoque asumido consiste en desarrollar un sistema de procesamiento de lenguaje natural basado en reconocimiento de entidades nombradas (NER), orientado a un dominio específico con un vocabulario determinado. Se trata de una solución simple, pero efectiva, centrada en reconocer un conjunto reducido de entidades relevantes.

2. Contexto

El reconocimiento de entidades nombradas (NER) es una tarea fundamental dentro del procesamiento de lenguaje natural que consiste en identificar y clasificar entidades relevantes dentro de un texto.

Una entidad nombrada es, en términos generales, cualquier elemento que pueda ser referido mediante un nombre propio: una persona, una ubicación o

una organización. La tarea de reconocimiento de entidades nombradas (*Named Entity Recognition*, NER) consiste en encontrar fragmentos de texto que constituyan nombres propios y etiquetar el tipo de entidad correspondiente (traducción propia de Jurafsky & Martin, 2024, sec. 11.5.1).

Pero existe una dificultad inherente: la ambigüedad. Por ejemplo, “puerta de entrada” podría significar “lugar” pero también “objeto”, dependerá del contexto si se está haciendo referencia a que *en la puerta de entrada* sucede determinado problema, en ese caso la entidad correcta es LUGAR, o si es *la puerta de entrada* el fenómeno problemático, entonces sería OBJETO. Por este motivo, la evaluación de los modelos de NER no es simple y requiere de diversas técnicas, como evaluar métricas de clasificación, visualizar predicciones en comparación con ejemplos reales, graficar proporción de falsos positivos y falsos negativos, utilizar el modelo en un entorno simulado (como a través de un bot de Telegram), entre otros.

Los sistemas de reconocimiento de entidades nombradas se evalúan mediante las métricas *recall*, *precision* y medida F1. El *recall* es la proporción entre la cantidad de respuestas correctamente etiquetadas y el total que debería haber sido etiquetado; la *precision* es la proporción entre la cantidad de respuestas correctamente etiquetadas y el total de respuestas etiquetadas; y la medida F1 corresponde a la media armónica entre ambas métricas (traducción propia de Jurafsky & Martin, 2024, sec. 11.5.4).

El reconocimiento de entidades nombradas combina dos desafíos simultáneos: la clasificación de entidades y la segmentación, es decir, la identificación precisa de los índices de inicio y fin de cada entidad dentro del texto. Esta característica introduce una complejidad adicional tanto en el entrenamiento como en la evaluación de los modelos, ya que errores pequeños en la delimitación de una entidad pueden generar múltiples errores de evaluación. Por ejemplo, si un modelo identifica únicamente “Jane” en lugar de “Jane Villanueva” como una entidad de tipo *persona*, se produce simultáneamente un falso positivo al segmentar incorrectamente la entidad y

un falso negativo al no reconocer el resto del span correspondiente (ejemplo adaptado de Jurafsky & Martin, 2024, sec. 11.5.4).

El hecho de que el etiquetado de entidades nombradas incluya un componente de segmentación, genera ciertos problemas en la evaluación. Es por esto que también es importante evaluar la presencia de falsos positivos y falsos negativos, ya que el entrenamiento de los modelos NER busca minimizar la diferencia entre las anotaciones reales y las predicciones realizadas por el modelo, tanto en la clasificación de la entidad como en la identificación correcta de sus posiciones de inicio y fin dentro del texto.

2.1 Criterios de éxito

En el contexto de este problema, un falso positivo (FP) ocurre cuando el modelo detecta una entidad inexistente o asigna incorrectamente una etiqueta a un fragmento del texto, mientras que un falso negativo (FN) sucede cuando el sistema no reconoce una entidad relevante presente en el mensaje del usuario. Maximizar la precisión implica reducir la cantidad de falsos positivos, priorizando que las entidades detectadas sean correctas, mientras que maximizar el recall busca reducir los falsos negativos, favoreciendo que la mayor cantidad posible de entidades relevantes sean identificadas por el modelo.

En el contexto del sistema propuesto, resulta relevante maximizar el recall del modelo, ya que los falsos negativos implican la pérdida de información al omitir entidades como el objeto afectado o la ubicación del problema. Aunque los falsos positivos pueden introducir ruido en la salida, generalmente permiten conservar suficiente información contextual para que el ticket siga siendo interpretable. Por este motivo, se priorizó un equilibrio entre precisión y recall, con especial atención en minimizar los falsos negativos, es decir, entidades omitidas durante la extracción.

3. Metodología

Para este proyecto, se evaluaron dos métodos de entrenamiento: aprendizaje por transferencia, donde se realizó un *fine-tuning* de dos modelos pre entrenados de

spaCy, *es_core_news_lg* y la versión pequeña *es_core_news_sm*, y el entrenamiento desde cero, en el cual entrenamos un modelo de Campos Aleatorios Condicionales (CRF). Para ello, se dividieron los datos en conjuntos de entrenamiento, validación y testeo, donde se evaluó el rendimiento de los modelos a lo largo del entrenamiento y sobre datos no vistos. Luego, se comparó la performance entre los modelos y se analizaron sus errores.

3.1 Conjuntos de datos

El conjunto de datos utilizado en este proyecto fue construido de forma sintética, dado que no existen disponibles en Internet conjuntos de datos anotados para esta tarea con vocabulario específico. Por lo tanto, con el objetivo de representar incidencias típicas que pueden ocurrir en aulas y espacios educativos, se incluyeron tanto ejemplos basados en situaciones reales como frases simuladas que reflejan el tipo de lenguaje natural que emplearían los usuarios al reportar problemas. Además, se agregaron ejemplos sin etiquetar, mensajes que no incluían ninguna entidad en su descripción, dado que los usuarios podrían escribir frases que no correspondan a ninguna incidencia, como así también mensajes que no tienen las tres entidades posibles (por ejemplo, OBJETO y PROBLEMA pero no explicitan LUGAR). La idea fue buscar que los modelos sean robustos ante estas situaciones. Se creó una cantidad de ejemplos y etiquetas de forma manual y posteriormente se utilizaron herramientas de Inteligencia Artificial para aumentar el tamaño del dataset y verificar el correcto etiquetado de los índices de las entidades.

El conjunto de datos consistió de 570 ejemplos anotados con tres tipos de entidades: OBJETO, que representa el elemento afectado (por ejemplo, “marcador”, “proyector”); PROBLEMA, que describe la incidencia (como “no funciona”, “está roto”); y LUGAR, que indica la ubicación donde ocurre el problema (por ejemplo, “aula 04-05” o “biblioteca”).

A continuación, una muestra del conjunto de datos empleado para este proyecto:

```
{"text": "El borrador de la pizarra está roto en el aula 02-04", "entities":  
[[3, 11, "OBJETO"], [26, 35, "PROBLEMA"], [42, 52, "LUGAR"]]}
```

```
{  
  "text": "No funciona el ac en el aula 01-03",  
  "entities": [[0, 11, "PROBLEMA"], [15, 17, "OBJETO"], [24, 34, "LUGAR"]]  
}  
  
{  
  "text": "queria avisar que en la biblioteca vi que el aire tira agua",  
  "entities": [[24, 34, "LUGAR"], [45, 49, "OBJETO"], [50, 59, "PROBLEMA"]]  
}  
  
{  
  "text": "Me podrías decir cuándo son las inscripciones a los finales?",  
  "entities": []  
}
```

Para crear los conjuntos de datos, en primer lugar, se realizó una verificación final del dataset para asegurar que no existieran desalineaciones entre los offsets de las entidades y la tokenización generada por spaCy. Para ello, se utilizó la función `offsets_to_biluo_tags()`, que permite comprobar si las entidades pueden representarse correctamente mediante el esquema BILUO. Los ejemplos inválidos fueron contabilizados para confirmar que el dataset estuviera completamente alineado antes del entrenamiento. El esquema BILOU significa:

- B (Begin): inicio de una entidad de múltiples tokens.
- I (Inside): parte interna de una entidad larga.
- L (Last): último token de la entidad.
- U (Unit): entidad de un único token.
- O (Outside): token que no pertenece a ninguna entidad.

Para entrenar y evaluar los modelos, el dataset fue dividido en tres conjuntos mediante la función `train_test_split()` de Scikit Learn, con una semilla de aleatoriedad para garantizar reproducibilidad y activando `shuffle=True` para mezclar los datos antes de dividirlos. La división resultante fue la siguiente: 80% para entrenamiento (456 ejemplos), 10% para desarrollo (57) y 10 % para validación (57). El conjunto de desarrollo (dev) se utilizó durante el entrenamiento para evaluar el desempeño del modelo en cada época, mientras que el conjunto de validación (val) se reservó para la evaluación final del modelo sobre datos no vistos.

Una vez definidos los subconjuntos, cada uno fue convertido al formato binario de spaCy mediante la clase `DocBin`. Para cada ejemplo, se creó un objeto `Doc` a partir

del texto utilizando `nlp.make_doc()`, y posteriormente se reconstruyeron las entidades anotadas mediante `doc.char_span()`. Este método de spaCy permite crear un segmento `span` utilizando los índices de los caracteres inicial y final del texto original en lugar de los índices de los tokens. Se aplica `alignment_mode="contract"` lo que garantiza que el `span` resultante sólo incluya tokens completamente contenidos dentro del rango de caracteres especificado, ajustando automáticamente `offsets` desalineados a límites válidos de tokens para evitar errores o entidades omitidas.

Finalmente, los documentos procesados fueron serializados en los archivos `train.spacy`, `dev.spacy` y `val.spacy` dentro de la carpeta `data/`.

3.2 Modelo 1: spaCy *es_core_news_lg*

es_core_news_lg es un pipeline de spaCy pre entrenado sobre textos periodísticos en español. Se trata de la versión grande del modelo, lo que implica que incluye `embeddings` estáticos pre entrenados.

Se eligió este modelo por tres razones principales. En primer lugar, al estar pre entrenado en español, sus representaciones capturan la morfología y semántica del idioma que serían imposibles de aprender desde cero con solo 570 ejemplos. En segundo lugar, la versión grande incorpora vectores de palabras estáticos pre entrenados que mejoran la capacidad de generalización ante palabras poco frecuentes o con errores tipográficos, situación habitual en mensajes escritos por usuarios de forma rápida. En tercer lugar, spaCy ofrece un pipeline de entrenamiento configurable y bien documentado.

La implementación consistió en tomar el pipeline base y reemplazar su componente `ner` por uno entrenado desde cero sobre el dataset del proyecto, conservando las representaciones vectoriales pre entrenadas del componente `tok2vec`. El entrenamiento se realizó mediante el comando `python -m spacy train` de spaCy, usando `train.spacy` como corpus de entrenamiento y `dev.spacy` para monitorear el desempeño por época. Al finalizar, spaCy guarda automáticamente el

modelo con mejor desempeño (score) en output/model-best, que es el que se usa para la evaluación final sobre el conjunto de validación.

3.2.1 Configuración del modelo

La configuración aplicada parte de la configuración base provista por spaCy para el modelo `es_core_news_lg`, generada mediante el comando `python -m spacy init fill-config`. Los valores por defecto de esta configuración están optimizados para tareas de NER en español sobre textos de dominio general, por lo que se decidió mantenerlos sin modificaciones. A continuación, detallamos la configuración aplicada al modelo para comprender cómo funciona un pipeline de spaCy:

Para la representación vectorial de los datos se utiliza la arquitectura `Tok2Vec.v2`, basada en una combinación de embeddings de subpalabras y vectores pre entrenados del modelo `es_core_news_lg`.

El bloque `MultiHashEmbed.v2` genera embeddings utilizando varios atributos de cada token: la forma normalizada (`NORM`), prefijos (`PREFIX`), sufijos (`SUFFIX`) y patrones ortográficos (`SHAPE`). Este enfoque permite capturar mejor la información morfológica de las entidades en lugar de quedarse con una única representación (como tokenizar y vectorizar palabras individuales).

La opción `include_static_vectors = true` incorpora los embeddings estáticos pre entrenados del modelo de spaCy. Las representaciones generadas se procesan mediante un codificador contextual `MaxoutWindowEncoder.v2`, configurado con una dimensión de tamaño 256, profundidad de 8 capas y una ventana de contexto de tamaño 1. Este encoder permite que cada token incorpore información de los tokens vecinos inmediatos, generando embeddings contextuales capaces de capturar dependencias locales dentro de la oración.

Sobre estas representaciones vectoriales opera el componente `ner`, implementado mediante la arquitectura `TransitionBasedParser.v2`. Este modelo aborda la tarea de NER como un problema secuencial de transición, aprendiendo

simultáneamente a clasificar entidades y determinar sus posiciones de inicio y fin dentro del texto. El clasificador interno está configurado con un tamaño oculto (`hidden_width`) de 64 neuronas y funciones Maxout de dos piezas (`maxout_pieces = 2`).

El entrenamiento aplica el optimizador Adam (`Adam.v1`) con una tasa de aprendizaje de 0.001, regularización L2 de 0.01 y gradient clipping de 1.0, con el objetivo de estabilizar el aprendizaje y reducir el riesgo de sobreajuste. Además, utiliza un dropout de 0.1, permitiendo desactivar aleatoriamente neuronas durante el entrenamiento para mejorar la capacidad de generalización del modelo. El entrenamiento se ejecuta durante un máximo de 20.000 pasos (`max_steps = 20000`), evaluando el desempeño cada 200 iteraciones (`eval_frequency = 200`). Finalmente, los batches de entrenamiento se generan dinámicamente mediante `spacy.batch_by_words.v1`, utilizando un esquema incremental (`compounding.v1`) que aumenta progresivamente el tamaño de batch desde 100 hasta 1000 palabras.

3.3 Modelo 2: CRF (Conditional Random Field)

Un Conditional Random Field (CRF) es un modelo probabilístico no dirigido para el etiquetado de secuencias. El CRF modela la probabilidad conjunta de toda la secuencia de etiquetas dado el texto de entrada, lo que le permite capturar dependencias entre etiquetas consecutivas. Mientras que un clasificador predice una etiqueta para una sola muestra sin considerar las muestras "vecinas", un CRF puede tener en cuenta el contexto.

Se eligió el CRF como modelo de comparación por ser el baseline clásico en tareas de NER previo a la era del Deep Learning. Su principal ventaja es que no requiere grandes volúmenes de datos para entrenarse y es muy interpretable, dado que las decisiones del modelo pueden rastrearse hasta las features individuales de cada token. Esto sirve como un punto de comparación para evaluar cuánto aporta el uso

de embeddings pre entrenados respecto a features léxicas simples. Para entrenar este modelo, el dataset fue convertido al esquema de etiquetado BIO.

En el etiquetado BIO, cualquier token que inicia una secuencia de interés se marca con la etiqueta B, los tokens que se encuentran dentro de dicha secuencia se etiquetan con I, y aquellos que no pertenecen a ninguna entidad se etiquetan con O. Este enfoque tiene la ventaja de permitir representar la tarea asignando una única etiqueta a cada palabra de entrada (traducción propia de Jurafsky & Martin, 2024, sec. 11.5.2).

La implementación se realizó con la librería *sklearn-crfsuite* de Scikit Learn. Cada token fue representado mediante un vector de features extraídas manualmente con la función `token_features()`, que extrae la palabra en minúscula, prefijos y sufijos de dos y tres caracteres, si el token está completamente en mayúsculas, si tiene formato de título, si es un dígito, y si contiene un guión. Además, para capturar el contexto local, se incorporaron las mismas features del token anterior y del siguiente (ventana de contexto de tamaño 1), junto con marcadores de inicio (BOS, *Begin Of Sentence*) y fin (EOS, *End Of Sentence*) de oración. Esta ventana de contexto permite al modelo detectar patrones como entidades precedidas por preposiciones ("en el aula") o problemas expresados como verbos negados ("no funciona").

En el entrenamiento, se emplearon los siguientes hiperparámetros: algoritmo de optimización L-BFGS, que permite un aprendizaje eficiente con convergencia rápida sobre datasets de tamaño reducido. Este optimizador, a diferencia de SGD o Adam, es más preciso porque explora todo el espacio de posibilidades al tomar todo el dataset en un único lote (algo solo posible en conjuntos de datos pequeños). Para controlar el sobreajuste, se aplicó una regularización L1 con coeficiente $c1=0.1$, que promueve la esparsidad en los pesos del modelo, y una regularización L2 con coeficiente $c2=0.1$, que penaliza pesos de gran magnitud. El entrenamiento se ejecutó durante un máximo de 200 iteraciones.

3.4 Modelo 3: spaCy *es_core_news_sm*

es_core_news_sm es la versión pequeña (*small*) del pipeline de spaCy para español, equivalente al modelo grande (*es_core_news_lg*) pero sin embeddings estáticos pre entrenados.

Se eligió este modelo para cuantificar el impacto del tamaño del modelo base sobre el resultado del *fine-tuning*. Al compartir exactamente la misma arquitectura y configuración de entrenamiento que el modelo grande, cualquier diferencia en las métricas finales puede atribuirse directamente a la calidad de las representaciones vectoriales pre entrenadas.

La implementación fue idéntica a la del modelo grande: se descargó la configuración base correspondiente a *es_core_news_sm*, se completó con `python -m spacy init fill-config`, y se entrenó con los mismos conjuntos `train.spacy` y `dev.spacy`, guardando el mejor modelo en `output_small/model-best`.

4. Experimentos y Resultados

4.1 Modelo 1: spaCy *es_core_news_lg*

A continuación, se muestran los resultados del entrenamiento.

Definiciones:

- **LOSS TOK2VEC**: error en la forma en que las palabras o tokens son convertidos en representaciones numéricas.
- **LOSS NER**: error en la identificación y etiquetado de entidades dentro del texto.
- **ENTS_P**: es la precisión sobre las entidades nombradas. Es decir, de todas las entidades que el modelo predijo, cuántas eran correctas.
- **ENTS_R**: es el recall sobre las entidades nombradas. Es decir, de todas las entidades reales, cuántas logró encontrar el modelo.

- **ENTS_F**: es la medida F1 de las entidades nombradas, que combina precisión y recall en un único valor.
- **SCORE**: es una métrica global utilizada por spaCy para seleccionar el mejor modelo durante el entrenamiento. Es el valor que determina qué *checkpoint* se guarda como model-best.

Resultados:

E	#	LOSS TOK2VEC	LOSS NER	ENTS_F	ENTS_P	ENTS_R	SCORE	
0	0	0.00	59.43	0.00	0.00	0.00	0.00	
4	200	44.57	3959.99	77.07	74.69	79.61	0.77	
10	400	48.73	1105.22	77.08	77.85	76.32	0.77	
17	600	45.12	721.62	81.05	80.52	81.58	0.81	
26	800	47.58	583.93	79.47	80.00	78.95	0.79	
37	1000	47.87	386.88	77.38	77.12	77.63	0.77	
50	1200	50.02	287.49	80.00	79.74	80.26	0.80	
67	1400	60.41	263.22	81.31	81.05	81.58	0.81	
86	1600	49.57	247.30	78.69	78.43	78.95	0.79	
110	1800	56.06	279.50	76.92	78.23	75.66	0.77	
139	2000	73.62	285.53	81.88	83.56	80.26	0.82	
175	2200	43.90	254.98	80.81	82.76	78.95	0.81	
215	2400	51.55	271.63	81.70	81.17	82.24	0.82	
255	2600	63.51	297.85	81.73	82.55	80.92	0.82	
295	2800	48.79	191.95	81.31	81.05	81.58	0.81	
335	3000	59.51	162.57	83.17	83.44	82.89	0.83	◀ MEJOR CHECKPOINT (model-best)
375	3200	36.58	156.75	77.89	78.15	77.63	0.78	
415	3400	176.58	229.80	80.00	79.74	80.26	0.80	
455	3600	116.41	232.83	78.55	78.81	78.29	0.79	
495	3800	73.05	175.52	75.82	75.32	76.32	0.76	
535	4000	19.77	138.44	78.00	79.05	76.97	0.78	
575	4200	146.85	188.31	76.47	75.97	76.97	0.76	
615	4400	65.61	175.23	80.77	78.75	82.89	0.81	
655	4600	168.39	206.27	77.33	78.38	76.32	0.77	

Observando las épocas de entrenamiento, se puede notar que la pérdida del componente NER (LOSS NER) disminuyó progresivamente, lo que indica que el modelo fue aprendiendo patrones relevantes a lo largo del proceso. El mejor desempeño se sucede alrededor de las 3000 iteraciones de entrenamiento, donde el score alcanza su valor máximo. En ese punto, el modelo logró un F1-score de 83.17, una precisión de 83.44 y un recall de 82.89. El alto valor de recall en este checkpoint evidencia un buen equilibrio entre la detección de entidades y la reducción de falsos positivos.

Resultados sobre el conjunto de validación:

```
=====
spaCy fine-tuneado (es_core_news_lg)
=====
```

Etiqueta	Precision	Recall	F1
PROBLEMA	0.6875	0.6735	0.6804
OBJETO	0.7308	0.7451	0.7379
LUGAR	0.7917	0.7600	0.7755
GLOBAL	0.7365	0.7267	0.7315

Los resultados finales muestran un desempeño relativamente equilibrado entre las distintas entidades. LUGAR alcanza el mejor desempeño con un F1-score de ~0.78 y una precisión de ~0.79, mientras que OBJETO también obtiene métricas satisfactorias (~0.74 de F1). Esto sugiere que cuando se describen elementos afectados y ubicaciones en los mensajes, suelen poseer patrones lingüísticos consistentes y fáciles de identificar por el modelo. Por otro lado, la entidad PROBLEMA fue la más difícil de reconocer, obteniendo el menor F1-score (~0.68). Esto puede atribuirse a la variabilidad lingüística con la que los usuarios describen incidencias o fallas, generando expresiones ambiguas y menos consistentes que cuando describen un lugar u objeto.

Resulta relevante destacar que las métricas de precision, recall y F1-score observadas en la curva de entrenamiento evaluadas iterativamente en el conjunto de testeo (0.8344 de precision, 0.8289 de recall y 0.8317 de F1) son más altas que las métricas globales sobre el conjunto de validación, aproximadamente un 10% de diferencia (promediando las diferencias entre las métricas de entrenamiento y validación). Con ello podemos inferir que el modelo sufrió un claro sobreajuste a los datos durante el entrenamiento.

4.2 Modelo 2: CRF (Conditional Random Field)

Resultados sobre el conjunto de validación:

```
=====
CRF (sklearn-crfsuite)
```

Etiqueta	Precision	Recall	F1
PROBLEMA	0.6939	0.6939	0.6939
OBJETO	0.7600	0.7451	0.7525
LUGAR	0.8298	0.7800	0.8041
GLOBAL	0.7603	0.7400	0.7500

Analizando cada categoría por separado, la entidad LUGAR presentó el mejor desempeño, alcanzando un F1-score de ~0.80. Por el contrario, la entidad PROBLEMA fue nuevamente la más difícil de identificar, obteniendo un F1-score de ~0.69. Esta dificultad puede explicarse por la mayor variabilidad lingüística y ambigüedad con la que los usuarios describen problemas en lenguaje natural. La entidad OBJETO mostró resultados sólidos con un F1-score de ~0.75, con valores balanceados entre precisión y recall. En comparación con el modelo spaCy lg, el CRF obtuvo resultados globales superiores en esta corrida, especialmente en precisión (0.7603 vs 0.7365 que obtuvo spaCy lg) y en la entidad LUGAR. Además, el recall de este modelo supera al de *es_core_news_lg* (0.7400 vs 0.7267 de spaCy lg).

4.3 Modelo 3: spaCy es_core_news_sm

Resultados del entrenamiento:

E	#	LOSS TOK2VEC	LOSS NER	ENTS_F	ENTS_P	ENTS_R	SCORE
0	0	0.00	59.43	0.00	0.00	0.00	0.00
4	200	43.84	3668.13	73.36	77.37	69.74	0.73
10	400	35.06	933.01	78.03	77.78	78.29	0.78
17	600	44.32	620.73	76.87	76.13	77.63	0.77
26	800	39.10	396.92	76.00	77.03	75.00	0.76
37	1000	61.25	350.01	75.00	75.00	75.00	0.75
50	1200	61.35	302.30	75.75	76.51	75.00	0.76
67	1400	71.15	264.93	75.50	76.00	75.00	0.75
86	1600	85.23	242.12	75.91	76.16	75.66	0.76
110	1800	81.37	278.86	73.65	75.69	71.71	0.74
139	2000	69.91	281.67	80.13	79.35	80.92	0.80 ← MEJOR CHECKPOINT (model-best)
175	2200	80.13	261.82	75.59	76.87	74.34	0.76
215	2400	99.40	250.37	74.07	75.86	72.37	0.74
255	2600	57.22	178.06	76.87	76.13	77.63	0.77
295	2800	96.64	186.35	74.75	76.55	73.03	0.75
335	3000	115.92	237.39	74.43	73.25	75.66	0.74
375	3200	102.61	165.46	74.75	74.51	75.00	0.75
415	3400	75.72	157.49	75.25	75.50	75.00	0.75
455	3600	37.60	137.56	75.82	75.32	76.32	0.76

Observando las épocas de entrenamiento, se puede notar que la pérdida del componente NER disminuyó progresivamente al igual que en el primer modelo de spaCy, indicando que el modelo logró aprender. El mejor desempeño se alcanzó alrededor de las 2000 iteraciones, donde el modelo mostró un F1-score de 80.13, un recall de 80.92 y una precisión de 79.35, evidenciando un equilibrio adecuado entre ambas métricas.

Resultados sobre el conjunto de validación:

spaCy fine-tuneado (es_core_news_sm)			
Etiqueta	Precision	Recall	F1
PROBLEMA	0.7000	0.7143	0.7071
OBJETO	0.7347	0.7059	0.7200
LUGAR	0.8163	0.8000	0.8081
GLOBAL	0.7500	0.7400	0.7450

Analizando cada categoría por separado, la entidad LUGAR presentó el mejor desempeño, alcanzando un F1-score de ~0.81 y una precisión de ~0.82. La entidad OBJETO también obtuvo resultados sólidos con un F1-score de ~0.72. Sin embargo, el desempeño más bajo volvió a observarse en la entidad PROBLEMA, con un

F1-score de ~0.71. Al igual que en los modelos anteriores, esta categoría resulta la más compleja debido a la diversidad de expresiones utilizadas para describir los problemas. En comparación con el primer modelo, *es_core_news_lg*, este modelo obtuvo resultados similares a nivel global (F1 0.7450 vs 0.7315 de spaCy lg), siendo la diferencia muy estrecha. Frente al modelo basado en CRF, spaCy sm quedó en segundo lugar en F1 exacto (0.7450 vs 0.7500 del CRF). Al igual que CRF, este modelo presenta mayor recall global que el modelo spaCy lg (tanto spaCy sm como CRF alcanzan 0.74 de recall global).

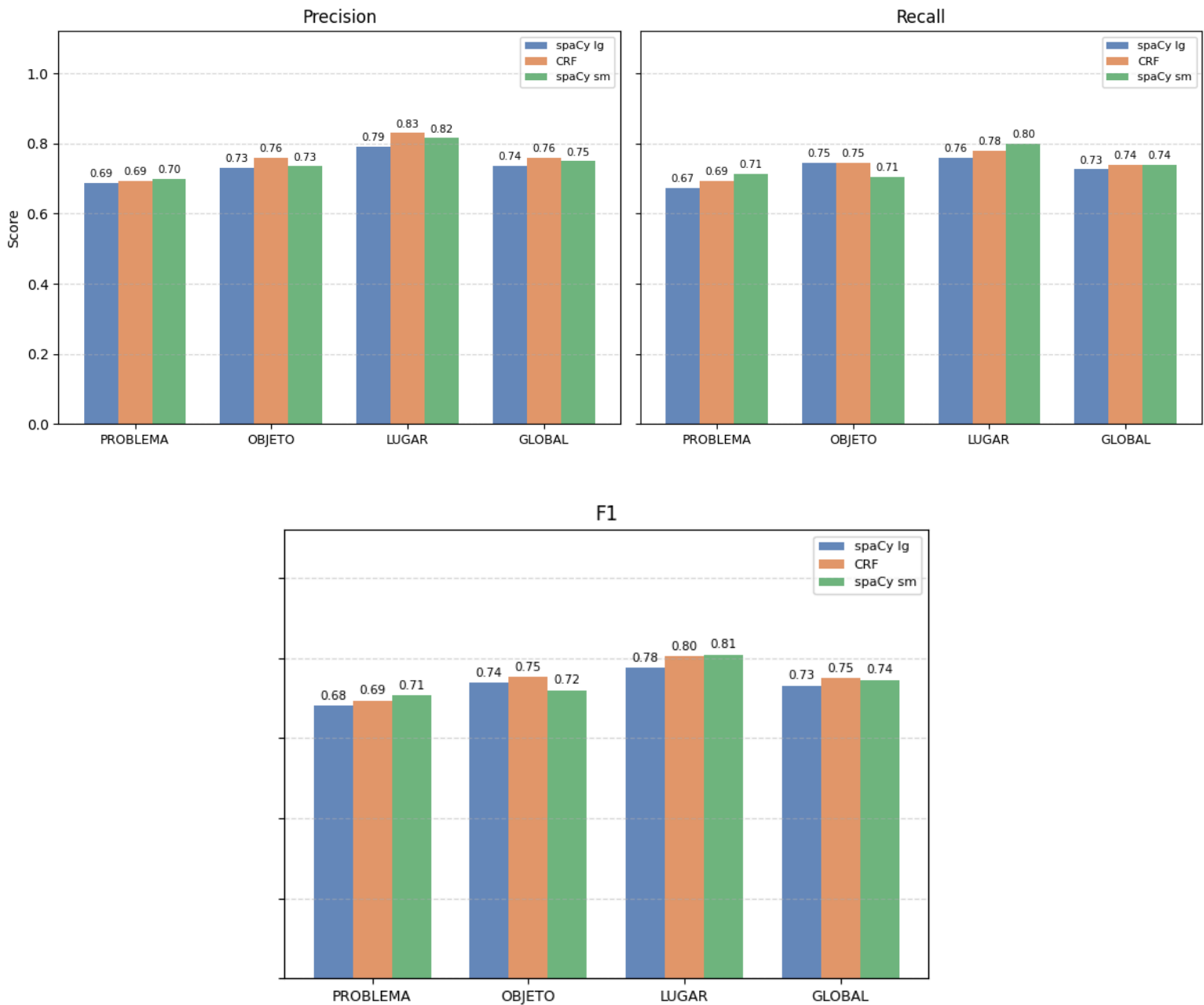
Se destaca nuevamente que las métricas de precision, recall y F1-score evaluadas iterativamente en el conjunto de testeo (0.7935 de precision, 0.8092 de recall y 0.8013 de F1) son más altas que las métricas globales sobre el conjunto de validación, aproximadamente 5.6% de diferencia (promediando las diferencias entre las métricas de entrenamiento y validación). Al igual que el modelo spaCy grande (*es_core_news_lg*), éste sufrió sobreajuste a los datos durante el entrenamiento, pero la mitad del overfitting observado en spaCy lg.

4.4 Comparación entre modelos

Resumen de métricas:

Modelo	P (global)	R (global)	F1 (global)
spaCy lg	0.7365	0.7267	0.7315
CRF	0.7603	0.7400	0.7500
spaCy sm	0.7500	0.7400	0.7450

Observando las métricas individuales, el modelo CRF obtiene los valores más altos de precisión global (0.76) y F1 global (0.75), seguido del modelo spaCy sm (F1=0.74) y en último lugar spaCy lg (F1=0.73). En cuanto al recall, CRF y spaCy sm empatan en el valor global más alto (0.74), dejando a spaCy lg en una posición desfavorable frente a los otros modelos. Se muestran las mismas métricas en formato de gráfico de barras:



Recordar que la métrica más importante de maximizar en este proyecto es el recall. En este sentido, observando la gráfica de recall, tanto spaCy sm (verde) como el modelo CRF (naranja) obtienen los mejores valores globales (0.74), empatando en primer lugar y dejando a spaCy lg (0.73) en una posición de desventaja. No obstante, observando la gráfica de F1, el CRF lidera globalmente (0.75) seguido muy de cerca por spaCy sm (0.74), mientras que en la gráfica de precisión el CRF también queda en primer puesto con 0.76.

Con estos resultados, el modelo Conditional Random Field supera a los otros dos en métricas exactas y además presenta el menor costo computacional (tanto en tiempo de entrenamiento e inferencia como en uso de memoria), aunque con la desventaja de que la extracción de features deba hacerse de forma manual, a diferencia de los modelos de spaCy que aprenden las características automáticamente. CRF es el baseline perfecto y, en esta corrida, también el mejor modelo en evaluación exacta.

4.5 Pruebas de integración

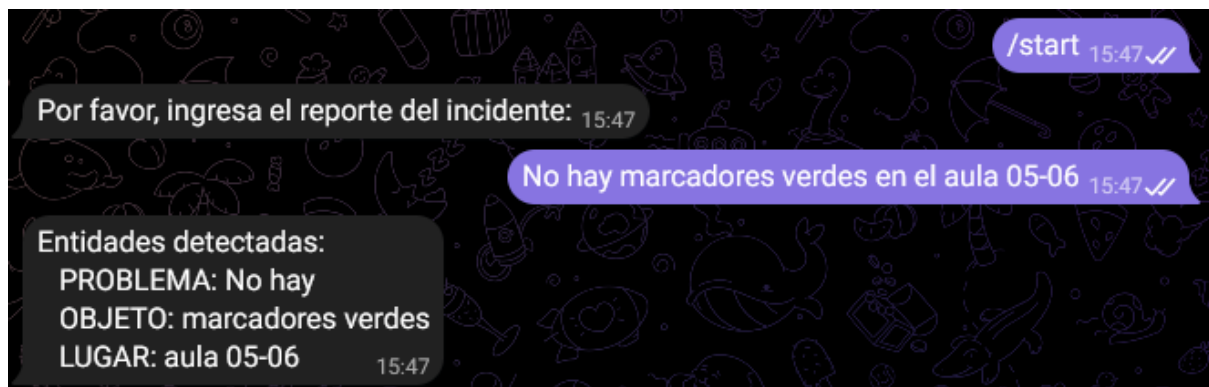
Se implementó un flujo de datos simple entre el bot de Telegram y el modelo spaCy re entrenado sobre *es_core_news_lg*, ambos alojados en un archivo de Google Colab. Este flujo consiste en extraer el problema del chat, preprocesar el mensaje, realizar la inferencia y, por último, se devuelven las entidades reconocidas a modo de demostración. Esta salida podría ser redirigida a otro medio, como a un sistema generador de tickets.

El preprocesamiento consistió en aplicar expresiones regulares para realizar una serie de eliminaciones de caracteres especiales y emojis, duplicados de guiones o espacios en blanco dejando solamente un carácter, de manera que el modelo realice la inferencia sobre una entrada limpia.

A continuación, se presentan cuatro escenarios de prueba:

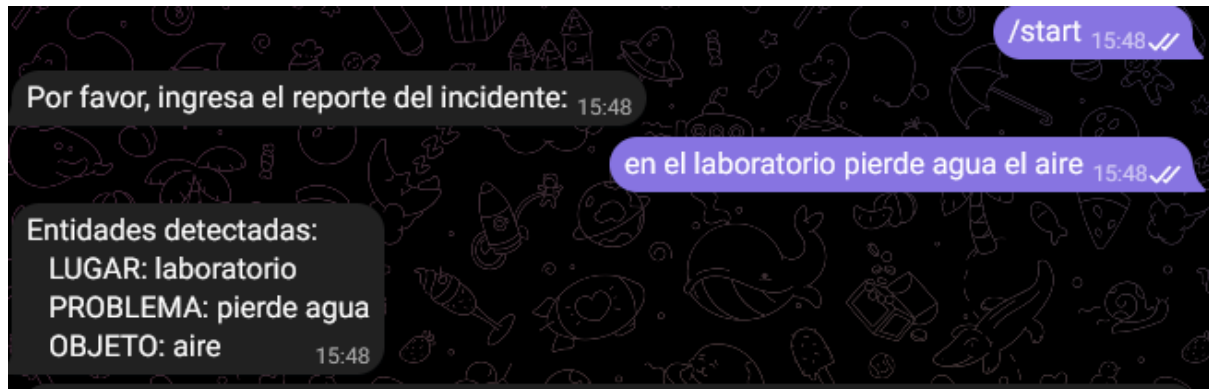
Escenario 1: Mensaje estructurado

"No hay marcadores verdes en el aula 05-06"



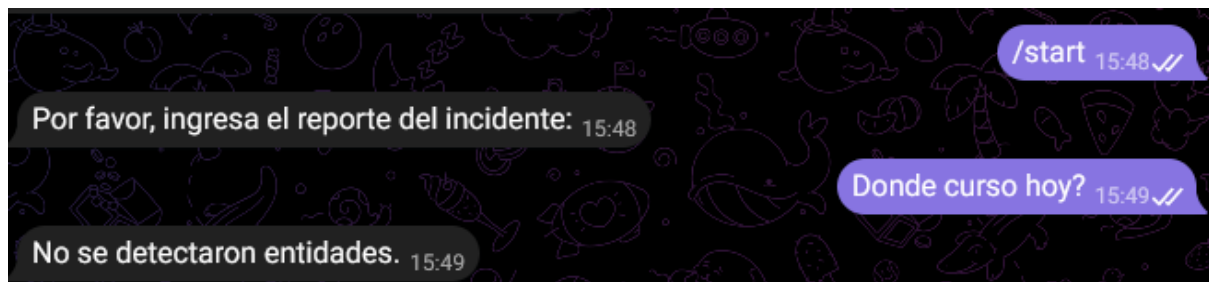
Escenario 2: Mensaje no estructurado

"en el laboratorio pierde agua el aire"



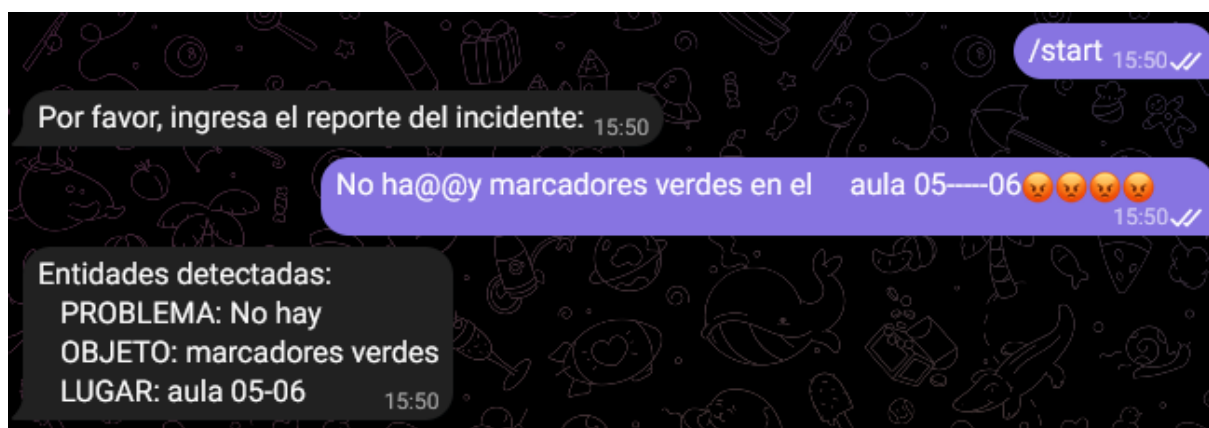
Escenario 3: Mensaje sin entidades

"Donde curso hoy?"



Escenario 4: Mensaje con caracteres especiales

"No ha@@@y marcadores verdes en el aula 05---06 🤔🤔🤔🤔"



4.6 Métricas con coincidencia parcial

Se recalcularon las métricas de precision, recall y F1-score de las predicciones de los modelos sobre el dataset de validación pero permitiendo una intersección parcial entre los índices de inicio y de fin del ejemplo verdadero en comparación con la predicción (siempre y cuando esté correctamente asignada la etiqueta de entidad al span detectado en la predicción). Esto permite tener métricas que penalizan menos fuertemente los modelos por no lograr predecir exactamente los índices pero que igual capturan una serie de tokens que, en conjunto y bien etiquetados, tienen sentido y son interpretables.

Se aplicó un umbral de 0.7, lo que significa que al menos el 70% de la entidad real debe coincidir con la predicción para considerar válido el resultado:

Modelo	Precision	Recall	F1-score
spaCy sm	0.8446	0.8333	0.8389
spaCy lg	0.8311	0.8200	0.8255
CRF	0.8288	0.8067	0.8176

En este escenario, el mejor modelo, aquel que maximiza el recall y presenta las métricas más altas, es el modelo chico de spaCy (*es_core_news_sm*), seguido de spaCy lg (*es_core_news_lg*) y en último lugar el modelo CRF, que anteriormente estaba en el primer puesto. En la sección 6.1 se explica la posible causa de este comportamiento.

5. Análisis de errores

En esta sección se analizan los errores cometidos por cada modelo sobre el conjunto de validación.

Para analizar el comportamiento de los errores modelos entrenados, se visualizó la cantidad de falsos positivos (FP) que cometen, predicciones de entidades que no están en el ejemplo real *gold*; cantidad de falsos negativos (FN), entidades que sí estaban en el ejemplo real pero que el modelo no detectó; y errores de tipo, cuando

identifica correctamente el span de la entidad (índices de inicio y de fin) pero asignan la etiqueta incorrecta. Recordar que el criterio de éxito de los modelos será aquel que minimice la proporción de falsos negativos.

Adicionalmente, se observaron los cinco ejemplos con más errores de cada modelo, coloreando las entidades predecidas del modelo en comparación con el ejemplo real.

5.1 Errores del Modelo 1: spaCy *es_core_news_lg*

FP, FN y Errores de tipo:

```
Falsos Positivos (FP): 39
Falsos Negativos (FN): 41
Errores de tipo      : 1
```

— FP más frecuentes —

```
2x esta manchada [PROBLEMA]
2x entrada [LUGAR]
2x portero [OBJETO]
1x canilla [OBJETO]
1x vidroi [OBJETO]
1x parque después de cursar [LUGAR]
1x reja de entrada [LUGAR]
1x candado roto [PROBLEMA]
1x rayas [PROBLEMA]
1x monitor de la pc [OBJETO]
```

— FN más frecuentes —

```
2x la entrada [LUGAR]
2x portero electrico [OBJETO]
1x la canilla [OBJETO]
1x un vidroi root [PROBLEMA]
1x reja [OBJETO]
1x entrada [LUGAR]
1x tiene el candado roto [PROBLEMA]
1x monitor [OBJETO]
1x tiene rayas [PROBLEMA]
1x se tildo [PROBLEMA]
```

— Errores de tipo —

```
'puerta de entrada' → predicho: LUGAR | gold: OBJETO
```

Total de errores (suma de falsos positivos y falsos negativos): 80

Se observa que el modelo spaCy lg comete más errores al detectar la entidad correcta en ejemplos que incluyen las frases "esta manchada", "entrada" y "portero", todas con dos ocurrencias en los falsos positivos. La ambigüedad de "entrada" y "portero" resulta lógica: "entrada" puede referirse a un LUGAR, como en "puerta de entrada", pero en un mensaje como "la puerta de entrada no funciona" estaría haciendo referencia a un OBJETO; "portero" puede aludir a un objeto, como "el portero eléctrico", o simplemente a una persona, en cuyo caso no tendría ninguna entidad asignable dentro del dominio definido ya que PERSONA no forma parte de las entidades consideradas en este proyecto. Por su parte, "esta manchada" representa un caso de ambigüedad contextual: dependiendo del mensaje, puede interpretarse como un PROBLEMA o no ser una entidad relevante. En esta corrida, la cantidad de falsos negativos (41) supera levemente a los falsos positivos (39), lo que indica que el modelo omite más entidades de las que detecta incorrectamente.

Los cinco ejemplos con más errores son:

Texto : vine a la facu y vi que la reja de entrada tiene el candado roto
Gold : [(27, 31, 'OBJETO'), (35, 42, 'LUGAR'), (43, 64, 'PROBLEMA')]
Pred : [(27, 42, 'LUGAR'), (52, 64, 'PROBLEMA')]
Predicción del modelo: vine a la facu y vi que la reja de entrada LUGAR tiene el candado roto PROBLEMA
Texto : vibracion excesiva en el motor del asensor del ala norte
Gold : [(0, 18, 'PROBLEMA'), (25, 42, 'OBJETO'), (47, 56, 'LUGAR')]
Pred : [(0, 18, 'PROBLEMA'), (25, 30, 'OBJETO'), (35, 56, 'LUGAR')]
Predicción del modelo: vibracion excesiva PROBLEMA en el motor OBJETO del asensor del ala norte LUGAR
Texto : los cables del rack de comunicaciones estan todos pelados
Gold : [(4, 10, 'OBJETO'), (15, 37, 'LUGAR'), (44, 57, 'PROBLEMA')]
Pred : [(4, 37, 'OBJETO')]
Predicción del modelo: los cables del rack de comunicaciones OBJETO estan todos pelados

Texto : el portero electrico de la entrada no da tono
 Gold : [(3, 20, 'OBJETO'), (24, 34, 'LUGAR'), (35, 45, 'PROBLEMA')]
 Pred : [(3, 10, 'OBJETO'), (27, 34, 'LUGAR'), (35, 45, 'PROBLEMA')]
Predicción del modelo: el portero OBJETO electrico de la entrada LUGAR no da tono PROBLEMA

Texto : el monitor de la pc del aula 04-05 tiene rayas
 Gold : [(3, 10, 'OBJETO'), (24, 34, 'LUGAR'), (35, 46, 'PROBLEMA')]
 Pred : [(3, 19, 'OBJETO'), (24, 34, 'LUGAR'), (41, 46, 'PROBLEMA')]
Predicción del modelo: el monitor de la pc OBJETO del aula 04-05 LUGAR tiene rayas PROBLEMA

Observar las siguientes situaciones:

1. Ambigüedad en el mensaje

“vine a la facu y vi que la reja de entrada tiene el candado roto”

Entidad	Ejemplo real	Predicción
OBJETO	reja	
PROBLEMA	tiene el candado roto	candado roto
LUGAR	entrada	reja de entrada

En este caso, el modelo entendió que la “reja de entrada” era el lugar del hecho, lo cual es comprensible dada la ambigüedad del mensaje pues si bien el ejemplo real dice que “reja” es el objeto y “entrada” el lugar, se puede tomar como que la frase entera hace referencia al lugar o el objeto afectado. Cualquier opción sigue siendo interpretable a la hora de generar un ticket estructurado así que por más que la predicción no coincida con el ejemplo real, se la puede considerar correcta.

2. Índices incorrectos pero resultado interpretable

“vibracion excesiva en el motor del asensor del ala norte”

Entidad	Ejemplo real	Predicción

OBJETO	motor del ascensor	motor
PROBLEMA	vibracion excesiva	vibracion excesiva
LUGAR	ala norte	asensor del ala norte

En este ejemplo la predicción también es interpretable y válida. Es más correcto indicar que el objeto afectado es el “motor del ascensor” que decir solo “motor”, pero también es válido decir que el lugar del hecho es en el “ascensor del ala norte”. Nuevamente, la predicción es correcta por más que difiera la serie de tokens capturados en las entidades OBJETO y LUGAR.

3. Frase rara

“los cables del rack de comunicaciones estan todos pelados”

Entidad	Ejemplo real	Predicción
OBJETO	cables	cables del rack de comunicaciones
PROBLEMA	todos pelados	
LUGAR	rack de comunicaciones	

En este ejemplo, “rack de comunicaciones” es una frase extraña, es entendible que el modelo no haya podido interpretar si se trataba de un objeto, problema o lugar. El ticket emitido con esta predicción no sería interpretable, dado que omite el problema asociado a los cables del rack.

4. Palabras omitidas pero resultado interpretable

“el portero electrico de la entrada no da tono”

Entidad	Ejemplo real	Predicción

OBJETO	portero electrico	portero
PROBLEMA	no da tono	no da tono
LUGAR	la entrada	entrada

Aquí, otra vez, la predicción es completamente interpretable a pesar de que el modelo haya omitido “electrico” en el objeto, lo cual no es grave pues al capturar correctamente el problema, un humano puede inferir que el objeto que “no da tono” es un portero eléctrico.

5. Predicción captura mejor las palabras

“el monitor de la pc del aula 04-05 tiene rayas”

Entidad	Ejemplo real	Predicción
OBJETO	monitor	monitor de la pc
PROBLEMA	tiene rayas	rayas
LUGAR	aula 04-05	aula 04-05

Finalmente, aquí vemos un ejemplo donde la predicción es mejor que el ejemplo real: es más interpretable indicar que el objeto afectado es el “monitor de la pc” que decir solo “monitor”. Luego, decir “tiene rayas” o “rayas” es igual de interpretable como el problema del mensaje.

Como se pudo observar, los modelos se ven fuertemente penalizados en sus métricas porque los índices de la serie de tokens que capturan no coinciden exactamente con los ejemplos reales del conjunto de datos. Sin embargo, las predicciones en muchos casos siguen siendo interpretables y válidas en el contexto de este proyecto.

5.2 Errores del Modelo 2: CRF

FP, FN y Errores de tipo:

Falsos Positivos (FP): 35

Falsos Negativos (FN): 39

Errores de tipo : 2

— FP más frecuentes (texto del span predicho incorrectamente) —

3x pc [OBJETO]
2x entrada [LUGAR]
2x no cierra [PROBLEMA]
1x no para [PROBLEMA]
1x canilla [OBJETO]
1x desbordado [PROBLEMA]
1x reja de entrada [LUGAR]
1x rayas [PROBLEMA]
1x todo bolcado [PROBLEMA]
1x tilde la pc [PROBLEMA]

— FN más frecuentes (entidad gold no detectada) —

2x la entrada [LUGAR]
2x portero electrico [OBJETO]
2x cerradura [OBJETO]
1x no para de perder [PROBLEMA]
1x la canilla [OBJETO]
1x un vidroi root [PROBLEMA]
1x esta desbordado [PROBLEMA]
1x reja [OBJETO]
1x entrada [LUGAR]
1x tiene rayas [PROBLEMA]

— Errores de tipo —

'cosina' → predicho: OBJETO | gold: LUGAR

'puerta de entrada' → predicho: LUGAR | gold: OBJETO

Total de errores (cantidad de FP y FN): 74.

En este caso, el error más frecuente del modelo es la detección incorrecta de "pc" como OBJETO (3 ocurrencias en FP), lo cual puede explicarse por la ambigüedad de este término en el contexto de los mensajes. Al igual que en el modelo spaCy lg, "entrada" también resulta problemática (2x en FP). Como en el primer modelo, las

entidades "la entrada" y "portero eléctrico" no fueron detectadas por el modelo en la mayoría de los casos (2x en FN cada una).

Los cinco ejemplos con más errores son:

Texto : Se callo la cortina en el salon de actos, el que esta al fondo Gold : [(3, 11, 'PROBLEMA'), (12, 19, 'OBJETO'), (26, 62, 'LUGAR')] Pred : [(0, 8, 'PROBLEMA'), (12, 19, 'OBJETO'), (26, 41, 'LUGAR'), (54, 62, 'PROBLEMA')] Predicción del modelo: Se callo PROBLEMA la cortina OBJETO en el salon de actos, LUGAR el que esta al fondo PROBLEMA
Texto : vine a la facu y vi que la puerta de entrada no cierra bien Gold : [(27, 44, 'OBJETO'), (45, 59, 'PROBLEMA')] Pred : [(27, 44, 'LUGAR'), (45, 54, 'PROBLEMA')] Predicción del modelo: vine a la facu y vi que la puerta de entrada LUGAR no cierra PROBLEMA bien
Texto : hola recien entre al bañp de hombres y la canilla no para de perder Gold : [(21, 36, 'LUGAR'), (39, 49, 'OBJETO'), (50, 67, 'PROBLEMA')] Pred : [(21, 36, 'LUGAR'), (42, 49, 'OBJETO'), (50, 57, 'PROBLEMA')] Predicción del modelo: hola recien entre al bañp de hombres LUGAR y la canilla OBJETO no para PROBLEMA de perder
Texto : vine a la facu y vi que la reja de entrada tiene el candado roto Gold : [(27, 31, 'OBJETO'), (35, 42, 'LUGAR'), (43, 64, 'PROBLEMA')] Pred : [(27, 42, 'LUGAR'), (43, 64, 'PROBLEMA')] Predicción del modelo: vine a la facu y vi que la reja de entrada LUGAR tiene el candado roto PROBLEMA
Texto : no hay luz en el aula 02-10 Gold : [(0, 10, 'PROBLEMA'), (17, 27, 'LUGAR')] Pred : [(0, 6, 'PROBLEMA'), (7, 10, 'OBJETO'), (17, 27, 'LUGAR')] Predicción del modelo: no hay PROBLEMA luz OBJETO en el aula 02-10 LUGAR

Analicemos algunas predicciones:

1. Entidad extra innecesaria

“Se callo la cortina en el salon de actos, el que esta al fondo”

Entidad	Ejemplo real	Predicción
OBJETO	cortina	cortina
PROBLEMA	callo la	Se callo
PROBLEMA		al fondo
LUGAR	salon de actos, el que esta al fondo	salon de actos,

Aquí se observa que el modelo detecta dos entidades PROBLEMA: “Se callo”, que es válida, y “al fondo” que es incorrecta. En este caso, si se genera un ticket estructurado quizás no sea tan interpretable de leer.

2. Índices bien capturados pero etiqueta incorrecta

“vine a la facu y vi que la puerta de entrada no cierra bien”

Entidad	Ejemplo real	Predicción
OBJETO	puerta de entrada	
PROBLEMA	no cierra bien	no cierra
LUGAR		puerta de entrada

En este caso, dada la ambigüedad de la frase “puerta de entrada”, la predicción difiere del ejemplo real, pero ambos podrían ser válidos: es correcto decir que el objeto afectado es la puerta de entrada como también entender que hace referencia a un lugar. Sin embargo, el ejemplo real es mejor dado que le da contexto al problema: permite interpretar que lo que “no cierra bien” es la puerta de entrada. En el ticket que generaría el modelo en base a su predicción, diría que *algo no cierra* en

la puerta de entrada, sin definir qué, pero igualmente es interpretable y puede considerarse válida esta predicción.

3. Índices incorrectos pero resultado interpretable

“hola recién entre al baño de hombres y la canilla no para de perder”

Entidad	Ejemplo real	Predicción
OBJETO	la canilla	canilla
PROBLEMA	no para de perder	no para
LUGAR	baño de hombres	baño de hombres

Aquí vemos nuevamente un caso donde los índices no coinciden entre la predicción y el ejemplo real, pero los fragmentos que el modelo captura y etiqueta son válidos e interpretables. Quizás el peor caso es que se genere un ticket donde el problema sea “no para”, dado que no explica qué es lo que no para.

4. Entidad extra innecesaria

“no hay luz en el aula 02-10”

Entidad	Ejemplo real	Predicción
OBJETO		luz
PROBLEMA	no hay luz	no hay
LUGAR	aula 02-10	aula 02-10

Observando esta situación, podemos inferir que lo que el modelo entiende que le sigue al PROBLEMA “no hay” suele ser un OBJETO. Pero, esta vez, “luz” formaba parte del problema. Aún así, el resultado sigue siendo interpretable.

5.3 Errores del Modelo 3: spaCy *es_core_news_sm*

FP, FN y Errores de tipo:

Falsos Positivos (FP): 37

Falsos Negativos (FN): 39

Errores de tipo : 1

— FP más frecuentes (texto del span predicho incorrectamente) —

2x esta manchada [PROBLEMA]
2x entrada [LUGAR]
2x portero [OBJETO]
2x no cierra [PROBLEMA]
1x canilla [OBJETO]
1x reja de entrada [LUGAR]
1x candado [OBJETO]
1x roto [PROBLEMA]
1x rayas [PROBLEMA]
1x monitor de la pc [OBJETO]

— FN más frecuentes (entidad gold no detectada) —

2x la entrada [LUGAR]
2x portero electrico [OBJETO]
1x la canilla [OBJETO]
1x reja [OBJETO]
1x entrada [LUGAR]
1x tiene el candado roto [PROBLEMA]
1x monitor [OBJETO]
1x tiene rayas [PROBLEMA]
1x pc [OBJETO]
1x se tildo [PROBLEMA]

— Errores de tipo —

'puerta de entrada' → predicho: LUGAR | gold: OBJETO

Total de errores (cantidad de FP y FN): 76.

Como en todos los casos, "entrada" y "portero" son incorrectamente detectados por el modelo (2x en FP cada uno). Además, este modelo presenta dificultades para identificar la entidad correcta de la frase "esta manchada" (2x en FP) y "no cierra" (2x en FP).

Los cinco ejemplos con más errores son:

Texto : vine a la facu y vi que la reja de entrada tiene el candado roto
Gold : [(27, 31, 'OBJETO'), (35, 42, 'LUGAR'), (43, 64, 'PROBLEMA')]
Pred : [(27, 42, 'LUGAR'), (52, 59, 'OBJETO'), (60, 64, 'PROBLEMA')]
Predicción del modelo: vine a la facu y vi que la reja de entrada LUGAR tiene el candado OBJETO roto PROBLEMA

Texto : vine a la facu y vi que la puerta de entrada no cierra bien
Gold : [(27, 44, 'OBJETO'), (45, 59, 'PROBLEMA')]
Pred : [(27, 44, 'LUGAR'), (45, 54, 'PROBLEMA')]
Predicción del modelo: vine a la facu y vi que la puerta de entrada LUGAR no cierra PROBLEMA bien

Texto : los cables del rack de comunicaciones estan todos pelados
Gold : [(4, 10, 'OBJETO'), (15, 37, 'LUGAR'), (44, 57, 'PROBLEMA')]
Pred : [(4, 43, 'OBJETO')]
Predicción del modelo: los cables del rack de comunicaciones estan OBJETO todos pelados

Texto : el portero electrico de la entrada no da tono
Gold : [(3, 20, 'OBJETO'), (24, 34, 'LUGAR'), (35, 45, 'PROBLEMA')]
Pred : [(3, 10, 'OBJETO'), (27, 34, 'LUGAR'), (35, 45, 'PROBLEMA')]
Predicción del modelo: el portero OBJETO electrico de la entrada LUGAR no da tono PROBLEMA

Texto : el monitor de la pc del aula 04-05 tiene rayas
Gold : [(3, 10, 'OBJETO'), (24, 34, 'LUGAR'), (35, 46, 'PROBLEMA')]
Pred : [(3, 19, 'OBJETO'), (24, 34, 'LUGAR'), (41, 46, 'PROBLEMA')]
Predicción del modelo: el monitor de la pc OBJETO del aula 04-05 LUGAR tiene rayas PROBLEMA

Observemos en detalle esta situación:

1. Ambigüedad en el mensaje

“vine a la facu y vi que la reja de entrada tiene el candado roto”

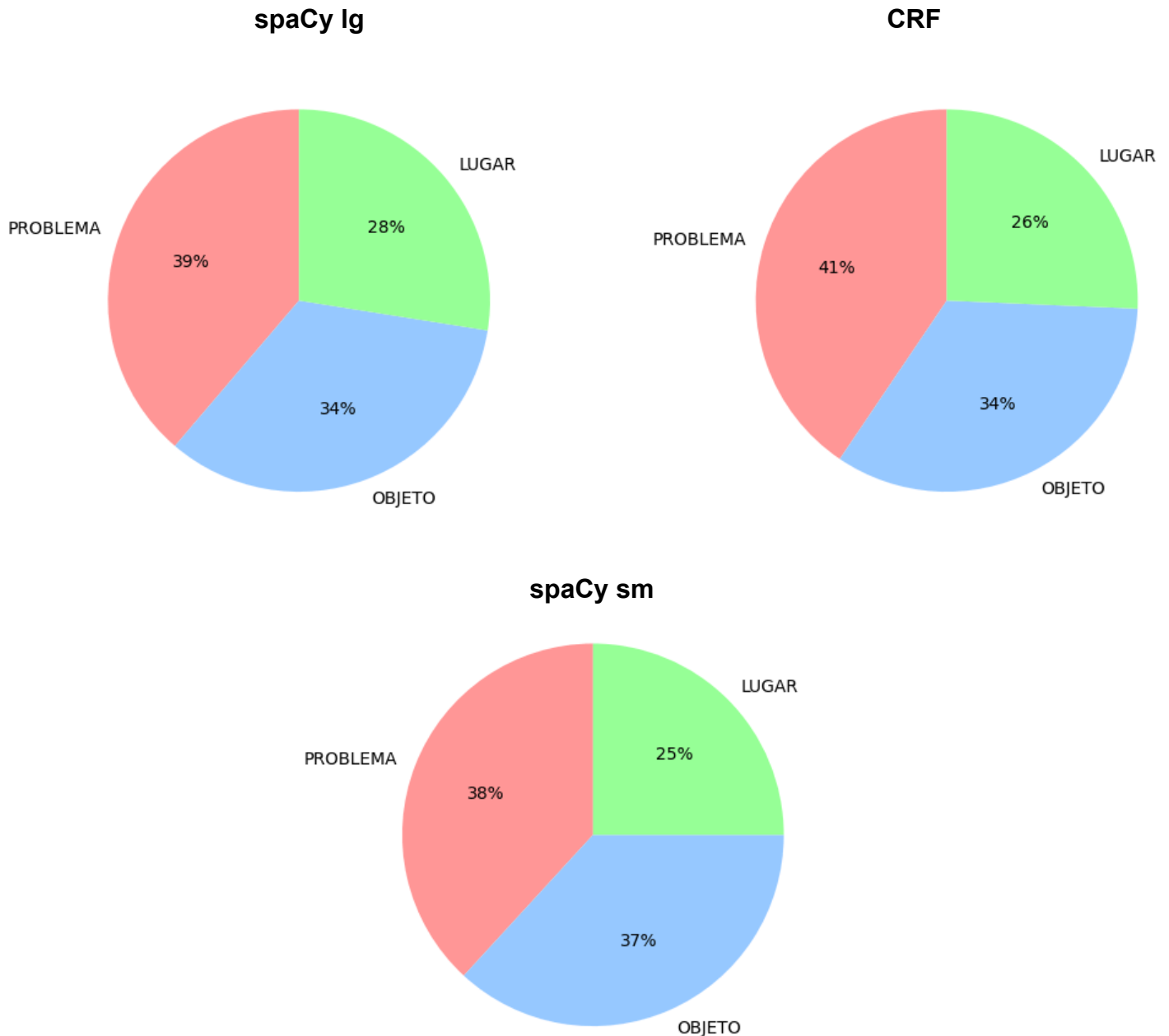
Entidad	Ejemplo real	Predicción
OBJETO	reja	candado
PROBLEMA	tiene el candado roto	roto
LUGAR	entrada	reja de entrada

Lo que captura el modelo como OBJETO y PROBLEMA es totalmente válido aunque la intención del usuario al escribir el mensaje era que el objeto afectado es la reja, no el candado, y el problema es que tiene el candado roto. La predicción es tan interpretable como el ejemplo real.

5.4 Distribución de errores por etiqueta

Suma de falsos positivos y negativos por etiqueta para cada modelo.

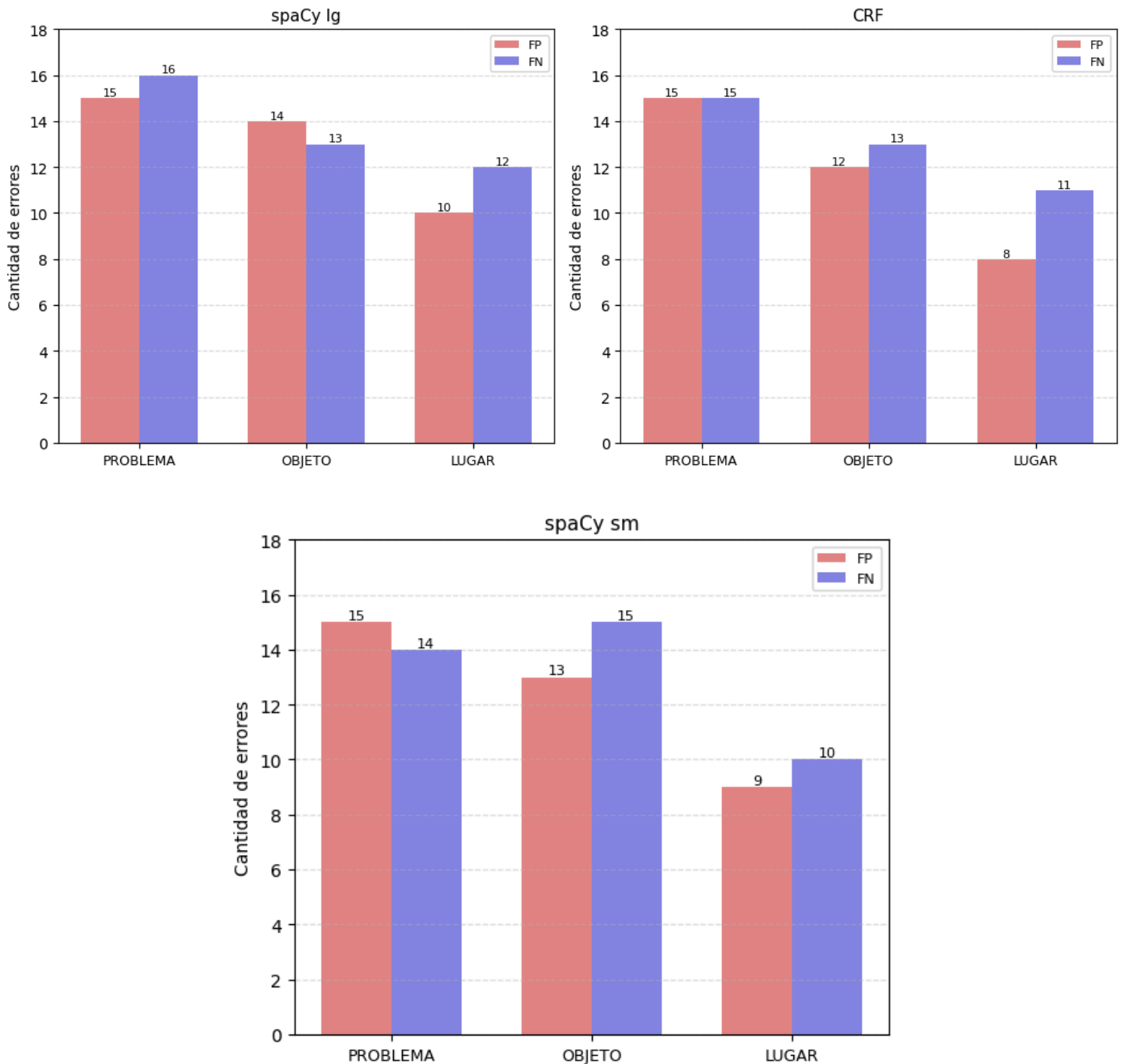
Gráfico de torta:



En los siguientes gráficos, se puede entender que la entidad que más problemas causa a los modelos es, irónicamente, la etiqueta PROBLEMA. Esto se explica por la ambigüedad y variabilidad lingüística y contextual en la que los usuarios pueden

describir incidencias. Mediante expresiones muy diferentes se puede describir lo mismo, como por ejemplo “no funciona” y “no anda”, como así también describir casos diferentes con alta variabilidad en las frases, como “sin tinta” o “pierde agua”. PROBLEMA es la entidad más difícil de segmentar y clasificar porque no cae en un patrón estructurado como sí sucede con LUGAR, que se observa que concentra la menor proporción de errores en todos los modelos. Esto puede deberse a que el lugar del hechos suele estar descrito como “aula 05-04”, “aula 02”, “biblioteca”, “laboratorio”, “entrada”, facilitando a los modelos capturar correctamente los índices y etiquetas de las secuencias. En cuanto a OBJETO, también se acerca a la proporción de errores de la entidad PROBLEMA para todos los modelos, lo que también se explica por la ambigüedad con la cual se expresan los objetos afectados, por ejemplo, con frases como “puerta de entrada” donde es difícil discernir si se trata de un lugar o un objeto.

Gráfico de barras:



Recordar que el objetivo de este proyecto, la métrica de éxito de los modelos, es la reducción de los falsos negativos, o lo que es igual, maximizar el recall. En los gráficos podemos observar que el modelo que concentra la menor cantidad de

falsos negativos son spaCy sm y CRF, ambos con 39 falsos negativos en total (se suma la cantidad de FN por entidad, para cada modelo), mientras que spaCy lg muestra 41 errores tipo II. Además, contando la cantidad de errores de cada modelo (realizando la suma del total de FP y FN), se observa que el modelo CRF concentra la menor proporción de errores (74 en total), seguido de spaCy sm (76) y en último lugar spaCy lg (80). Esto hace a los modelos más simples muy competitivos en este proyecto, dado que con menos recursos se consiguen mejores resultados (spaCy lg es un modelo más grande y, por ende, más pesado a comparación de su versión pequeña y la opción probabilística CRF).

6. Conclusiones

6.1 Resultados de los modelos

En términos generales, los tres modelos evaluados lograron resultados satisfactorios para la extracción automática de entidades dentro del dominio propuesto, aunque con diferencias claras en su capacidad de generalización y detección de entidades relevantes.

Por un lado, el modelo basado en CRF obtuvo el mejor desempeño global en métricas exactas ($F1=0.7500$), destacándose especialmente en precisión (0.7603) y en la entidad LUGAR ($F1=0.8041$). El modelo spaCy *es_core_news_sm* quedó en segundo lugar en métricas exactas ($F1=0.7450$), igualando el recall del CRF. Finalmente, el modelo spaCy entrenado sobre *es_core_news_lg* obtuvo el menor desempeño global ($F1=0.7315$), aunque con un recall de 0.7267 que sigue siendo competitivo.

Por otro lado, bajo el criterio de evaluación de coincidencia parcial, los modelos de spaCy superaron al CRF, en especial el modelo spaCy sm que superó al resto con un F1-score de 0.8389 . Esto sugiere que los enfoques poseen fortalezas diferentes. El modelo CRF demostró una mayor precisión en la delimitación exacta de las entidades, probablemente debido a que las entidades del dataset presentan patrones estructurados y repetitivos, lo que favorece a modelos estadísticos

basados en features locales. En cambio, los modelos de spaCy, aunque cometían pequeños errores en los índices de inicio y fin de las entidades, lograban identificar correctamente la región semántica correspondiente gracias a su capacidad de representación contextual. Bajo coincidencia exacta, estos pequeños desvíos eran penalizados como errores completos; sin embargo, el criterio de coincidencia parcial permitió reconocer predicciones con superposición significativa respecto de la entidad real. Como resultado, los modelos neuronales mostraron una mejor capacidad de detección semántica aproximada, mientras que el CRF conservó una ventaja en precisión estructural y delimitación exacta de spans.

En conclusión, considerando la evaluación de coincidencia exacta, el modelo CRF resulta el más adecuado para utilizar en producción por su mejor desempeño de métricas y su menor costo computacional. No obstante, si se prioriza la robustez ante pequeños errores de segmentación, el modelo spaCy *es_core_news_sm* representa una alternativa muy sólida al liderar en la evaluación de coincidencia parcial, en especial si el dominio del vocabulario escala, pues este modelo sugiere un potencial de generalización mayor, dado que entrenar un modelo CRF implica un proceso artesanal de extracción de características frente al pipeline sofisticado que ofrece spaCy donde las representaciones vectoriales se aprenden automáticamente.

6.2 Limitaciones

Entendemos que una de las principales limitaciones de este proyecto se encuentra en el conjunto de datos utilizado para el entrenamiento y evaluación de los modelos. Al tratarse de un dataset sintético y de volumen limitado, las expresiones incluidas no representan completamente la diversidad lingüística que podría aparecer en un entorno real de uso. En producción, los usuarios podrían emplear expresiones coloquiales, abreviaciones, errores ortográficos, mensajes poco estructurados, distintas formas de describir un mismo problema, emplear múltiples oraciones, información redundante o descripciones más extensas del incidente, incrementando la complejidad de la tarea de segmentación y clasificación de entidades, afectando la capacidad de generalización del sistema. No obstante, durante la construcción del dataset se intentó incorporar cierta variabilidad lingüística, incluyendo diferentes

formas de expresar incidencias, mensajes más informales e incluso ejemplos sin entidades etiquetadas, con el objetivo de aproximar parcialmente las condiciones reales de utilización del sistema.

Se podría explorar el rendimiento de los modelos spaCy haciendo cambios en sus hiperparámetros para evitar el drástico sobreajuste que presentaron, pero consideramos que la solución principal sería el aumento del tamaño y la complejidad del dataset. Además, el CRF funcionó como un modelo baseline presentando un rendimiento satisfactorio, por lo que no sería pertinente implementar modelos más complejos cuando el que presenta menor costo computacional funciona efectiva y eficientemente.

6.3 Próximos pasos

Como trabajo futuro, el primer paso sería aumentar el tamaño del dataset y, en lo posible, incorporar ejemplos de situaciones reales de uso.

Posteriormente, podría explorarse el uso de modelos basados en arquitecturas Transformer, como BERT, capaces de generar embeddings contextuales mediante mecanismos de *self-attention*. A diferencia de los enfoques utilizados en este trabajo, estos modelos construyen representaciones considerando simultáneamente el contexto completo de la secuencia, permitiendo capturar relaciones semánticas y dependencias lingüísticas más complejas. Esto podría servir para detectar entidades en mensajes ambiguos o con alta variabilidad lingüística, como PROBLEMA, donde la interpretación depende fuertemente del contexto.

Finalmente, también podría evaluarse el uso de modelos de lenguaje de gran escala (LLMs) para realizar tareas de extracción de entidades mediante *prompting* e *in-context learning*. En este enfoque, el modelo recibe ejemplos de entradas y salidas esperadas dentro del prompt, permitiéndole adaptarse dinámicamente al dominio sin necesidad de reentrenamiento explícito. Esto podría facilitar la incorporación rápida de nuevos patrones, vocabulario o tipos de incidentes observados en producción, especialmente en este escenario donde resulta difícil generar grandes volúmenes de datos anotados.

6.4 Pregunta final

¿Por qué un modelo neuronal, complejo y potente tiene peor rendimiento que un modelo estadístico simple?

Consideramos que esta pregunta resume el análisis realizado en el presente trabajo y que puede derivar en un debate de respuestas interesantes.

7. Recursos consultados

Explosion AI. (s.f.). *Spanish models es_core_news_lg*. spaCy.

https://spacy.io/models/es#es_core_news_lg

Explosion AI. (s.f.). *Spanish models es_core_news_sm*. spaCy.

https://spacy.io/models/es#es_core_news_sm

Explosion AI. (s.f.). *Tok2Vec*. spaCy API Reference. <https://spacy.io/api/tok2vec>

Explosion AI. (s.f.). *EntityRecognizer*. spaCy API Reference.

<https://spacy.io/api/entityrecognizer>

Explosion AI. (s.f.). *Corpus*. spaCy API Reference. <https://spacy.io/api/corpus>

Explosion AI. (s.f.). *Doc*. spaCy API Reference. <https://spacy.io/api/doc>

Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3rd ed., draft, cap.

11). Stanford University. https://web.stanford.edu/~jurafsky/slp3/old_aug24/11.pdf

Wikipedia contributors. (s.f.). *Conditional random field*. Wikipedia.

https://en-wikipedia-org.translate.goog/wiki/Conditional_random_field?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc